

■■ NeuroForge Backend Screen-Sharing Presentation Script

Step-by-Step Live IDE Code Walkthrough & Postman Demonstration Guide

1. PROJECT INTRODUCTION (Opening Statement)

What to say before opening the code in your IDE:

"Good morning/afternoon! Today I am presenting the backend for **NeuroForge**, an Enterprise Software Development Lifecycle (SDLC) Management Platform.

The backend handles user authentication, project tracking, agile sprint planning, task assignments, bug tracking, CI/CD build pipelines, and deployment logs.

For the technology stack, I used **Java 26** with **Spring Boot 4.1.0**. For database storage, I configured a dual-profile database setup: a local in-memory H2 database for offline testing, and a production **PostgreSQL 17.6** database hosted on **Supabase**.

Security and authentication are handled using **Spring Security**, **BCrypt** password hashing, and **stateless JSON Web Tokens (JWT)**.

I also integrated **Springdoc OpenAPI 3.0**, which automatically generates interactive **Swagger UI** documentation at <http://localhost:8080/swagger-ui/index.html>.

I will now share my screen and walk you through the actual source code structure in my IDE."

2. PROJECT STRUCTURE

STEP 1 — OPEN: IDE Project Explorer (Left Sidebar)

POINT TO: Package com.example.demo and sub-packages (Controller, config, model, repository, security, service)

"Here in the IDE, you can see our complete package structure under `com.example.demo`. We structured the project following the 3-tier REST architecture:

1. **Controller**: Contains 19 REST Controllers that expose HTTP endpoints for all SDLC modules.
2. **Service**: Contains 18 Service classes where all business logic and data access scoping reside.
3. **Repository**: Contains 18 Spring Data JPA Repository interfaces that handle database queries.
4. **Model**: Contains 19 JPA Entity classes mapped directly to PostgreSQL database tables.
5. **Security & Config**: Contains our JWT filters, security configuration, and database initializer beans."

3. BUILD CONFIGURATION

STEP 2 — OPEN: pom.xml

POINT TO: Parent version 4.1.0, spring-boot-starter-data-jpa, spring-boot-starter-security, postgresql, springdoc-openapi-starter-webmvc-ui

"In our `pom.xml` build configuration, we are using Spring Boot 4.1.0 compiled with Java 26.

- **Spring Boot Starter Data JPA**: Used for object-relational mapping with Hibernate.
- **Spring Boot Starter Security**: Manages authentication and request filtering.

- **PostgreSQL Driver:** Allows Spring Boot to connect to our PostgreSQL database on Supabase.
- **Springdoc OpenAPI UI:** Automatically generates interactive API documentation at `/swagger-ui/index.html`.

4. APPLICATION STARTUP

STEP 3 — OPEN: DemoApplication.java

POINT TO: `@SpringBootApplication` and `main()` method

"This is `DemoApplication.java`, the entry point of our backend.
The `@SpringBootApplication` annotation triggers Spring Boot's auto-configuration, component scanning across all sub-packages, and bootstraps the embedded Apache Tomcat web server on port 8080."

5. CONFIGURATION & DUAL-PROFILE DATABASE SETUP

STEP 4 — OPEN: `application.properties` AND `application-supabase.properties`

POINT TO: `spring.profiles.active=supabase` and
`jdbc:postgresql://db.lpwyljljovcpaaemlwdu.supabase.co:5432/postgres`

"Our project uses a dual-profile database setup.
When `spring.profiles.active` is set to `supabase`, Spring Boot connects directly to our production PostgreSQL 17 database hosted on Supabase using `PostgreSQLDriver`.
If we switch the profile to `local`, the backend connects to an in-memory H2 database for offline local testing."

6. SECURITY ARCHITECTURE

STEP 5 — OPEN: SecurityConfig.java

POINT TO: `PasswordEncoder`, `permitAll` matchers (`/api/auth/**`, `/swagger-ui/**`), `authenticated GET` matchers, `addFilterBefore(jwtFilter)`

"In `SecurityConfig.java`, we configured stateless session management.
• **BCryptPasswordEncoder:** Encrypts passwords using BCrypt salt before saving to the database.
• **Public Endpoints:** `/api/auth/**` (login/register) and `/swagger-ui/**` are permitted without authentication.
• **Read Access:** Any authenticated user with a valid JWT token can make GET requests.
• **Write Access:** POST, PUT, and DELETE requests on core resources require ADMIN or PROJECT_MANAGER roles.
• **Filter Order:** Incoming requests pass through `JwtFilter` before Spring Security's authentication filter."

STEP 6 — OPEN: `JwtUtil.java` AND `JwtFilter.java`

POINT TO: `JwtUtil.builder` (HS384 signature) and `JwtFilter.doFilterInternal`

"`JwtUtil.java` handles token creation and validation using HMAC-SHA384 (HS384).
`JwtFilter.java` intercepts every HTTP request. It checks for the Authorization: Bearer token header, extracts username and user role, and registers authentication into Spring's `SecurityContext`."

7. AUTHENTICATION & AUTOMATED SEEDING

STEP 7 — OPEN: `DataInitializer.java` AND `AuthController.java`

POINT TO: `DataInitializer.run()` admin seeding and `AuthController.login()`

"`DataInitializer` implements `CommandLineRunner` to seed default demo admin accounts (`admin@neuroforce.com` with password `password123`) into PostgreSQL on startup if missing.
In `AuthController.java`, when a user calls `POST /api/auth/login`, `AuthenticationManager` verifies the BCrypt password. If valid, `JwtUtil` generates a signed JWT token and returns it in the response."

8. AUTHORIZATION & DATA SCOPING

STEP 8 — OPEN: ProjectScopeResolver.java

POINT TO: ADMIN role check and teamMemberRepository lookup

"Beyond Spring Security role checks, we implemented row-level multi-tenancy security in ProjectScopeResolver. If the user has ROLE_ADMIN, access is unrestricted. If the user is a regular developer, it queries team_member to resolve only the project IDs where they are an assigned team member, preventing unauthorized data access."

9. CONTROLLER LAYER (PROJECT MODULE)

STEP 9 — OPEN: ProjectController.java

POINT TO: @RequestMapping(/api/projects), @GetMapping, @GetMapping("/{id}), @PostMapping

"This is ProjectController.java, mapped to /api/projects. It handles REST endpoints for SDLC projects:

- GET /api/projects: Fetches all projects.*
- GET /api/projects/{id}: Fetches a single project by ID.*
- POST /api/projects: Accepts a JSON payload and creates a new project.*

The controller does not write SQL; it delegates all logic to ProjectService."

10. SERVICE LAYER

STEP 10 — OPEN: ProjectService.java

POINT TO: getAllProjects() and getProjectById(Long id) scoping check

"Here in ProjectService.java:

- In getAllProjects(), it consults ProjectScopeResolver. If the user is Admin, it calls projectRepository.findAll(). If the user is a developer, it calls projectRepository.findByProjectIdIn(pids).*
- In getProjectById(id), if the user tries to access a project ID outside their assigned scope, it throws an HTTP 403 Forbidden exception."*

11. REPOSITORY LAYER

STEP 11 — OPEN: ProjectRepository.java

POINT TO: extends JpaRepository and findByIdIn()

"ProjectRepository is an interface extending JpaRepository. This provides built-in CRUD operations like findAll(), findById(), and save(). Spring Data JPA also automatically generates the SQL for custom queries like findByIdIn()."

12. MODEL / ENTITY LAYER

STEP 12 — OPEN: Project.java

POINT TO: @Entity, @Table(name = 'project'), @Id, @GeneratedValue(strategy = GenerationType.IDENTITY)

"This is the Project entity class mapped to database table project. @Id and @GeneratedValue(strategy = GenerationType.IDENTITY) map projectId to PostgreSQL auto-incrementing primary key columns. @NotBlank validates that name fields are not empty."

13. DATABASE & DTO ARCHITECTURE NOTE

EXPLANATION TO SPEAK:

"For database persistence, Hibernate maps Java Entities directly to PostgreSQL tables in Supabase.

Regarding DTOs: *In our current implementation, controllers directly accept JPA entity objects in @RequestBody payloads (with LoginRequest as an exception in AuthController). Reusing entities allowed rapid prototyping, and refactoring to dedicated DTOs with MapStruct is planned for future iterations."*

14. LIVE DEMONSTRATION (SWAGGER UI / POSTMAN)

Recommended Module: Project Management (/api/projects)

Operation	Endpoint	Payload / Action	Expected Result
1. Login	POST /api/auth/login	{ "email": "admin@neuroforce.com", "password": "password" }	200 OK (JWT Token)
2. Authorize	Swagger Authorize 	Paste Bearer Token	Unlocks Protected APIs
3. Get Projects	GET /api/projects	Header: Authorization Bearer	200 OK (16 Supabase Projects)
4. Get By ID	GET /api/projects/12	Header: Authorization Bearer	200 OK (Tester Project X)
5. Create Project	POST /api/projects	{ "name": "Demo Project", "status": "ACTIVE" }	200 OK (Generated ID)

Script to speak during live demo:

"Now I will demonstrate our live backend endpoints.

First, I send a POST request to /api/auth/login with admin credentials. The backend verifies the password with BCrypt and returns a signed JWT token.

Next, I copy the token and authorize our session.

Now I execute GET /api/projects. The request passes through JwtFilter, reaches ProjectController, calls ProjectService, and retrieves all 16 projects directly from our cloud Supabase PostgreSQL database.

When I query GET /api/projects/12, ProjectService verifies access scoping and returns 'Tester Project X'.

Finally, when I send a POST request to create a project, the record is saved permanently to PostgreSQL on Supabase and returns the created object with its new generated ID. Thank you!"